

Robinhood Agentic Trading — Dev Team Handoff

Prepared: August 16, 2026 **Scope:** Research and integration reference for connecting an AI agent to Robinhood's Agentic Trading MCP for equities and options.

Important notes before you start

- **Do the cloning, review, and running on your own hardware and time**, consistent with applicable workplace policy. This is personal financial activity.
 - **Guardrails live in code, not in prompts.** Community skills are tools the agent *can* call; the limits that protect your money must sit in your own code, outside the model's control.
 - **Verify each repo before depending on it.** Confirm the repo exists, read its license, and review its code. Several community repos use Robinhood's private/reverse-engineered API rather than the official MCP — prefer the official endpoint.
 - **Risk disclosure (Robinhood's own):** Agent trading can lose your entire deposit and may be hard to stop in real time. Options add leverage and time decay. This document is not financial advice.
-

1. The product

Robinhood **Agentic Trading**, in beta since May 27, 2026. A US user connects an AI agent to Robinhood's MCP server so it can place trades on the user's behalf. It runs on the Model Context Protocol (MCP), not a REST API. Equities and options are live now; crypto was added July 20, 2026. Futures, event contracts, and prediction markets were announced as coming but were not confirmed live as of last research.

Architecture facts to design around:

- Trades run in a dedicated agentic account, separate from your main portfolio. The agent only accesses funds you deposit there.
 - Push notification on every trade, plus real-time activity feed and P&L in the app.
 - Agents preview orders before placing so you can review details first.
 - Account creation and agent authentication are desktop-only.
 - Requires a primary Robinhood individual investing account in good standing first. Up to 10 self-directed individual accounts allowed, including the agentic one.
-

2. The connection (official path)

One command for Claude Code:

```
claude mcp add robinhood-trading --transport http
https://agent.robinhood.com/mcp/trading
```

Then `/mcp` → select `robinhood-trading` → authenticate.

- Endpoint is HTTP-transport MCP.
- Auth is OAuth completed on desktop — the step people most often get stuck on. Expect to reload the client and finish the OAuth flow in a desktop browser.
- Any MCP-compatible client works via the same URL. Officially supported: Claude Desktop, Claude Code, ChatGPT, Codex, Codex CLI, Cursor, Grok.
- Authoritative doc: `robinhood.com/us/en/support/articles/agent-trading-overview/`

Key point: The connection is trivial. What you're actually building is the **logic/guardrail layer** wrapped around the tools the MCP exposes (portfolio reads, quotes, order placement).

3. Repos to clone (skills for the trading brain)

```
git clone https://github.com/kevin1chun/robinhood-for-agents
git clone https://github.com/Open-Agent-Tools/open-stocks-mcp
git clone https://github.com/zaydiscolld/robinhood-cli-mcp-api
```

What each brings:

- **kevin1chun/robinhood-for-agents** — The skill layer itself: 50 tools plus guided workflows for auth, portfolio, research, trading, and options. Primary skill scaffold.
- **Open-Agent-Tools/open-stocks-mcp** — Production architecture reference: HTTP transport, Docker, structured error handling, multi-broker, options trading validated live. Study for robustness.
- **zaydiscolld/robinhood-cli-mcp-api** — Safety-engineering patterns: writes-gated design, dedup window, ref-ID idempotency for safe retries, "order history is the only proof" principle.

Optional zero-risk starting point:

```
git clone https://github.com/verygoodplugins/robinhood-mcp
```

Read-only. Explore the tool surface before anything can place an order.

4. Claude Code audit + integration prompt

Paste this after cloning the opiontrading repo.

You are auditing and helping integrate a live-money options trading system. Do NOT place any trades, run any live orders, or execute anything against a funded account during this session. Work in read-only/analysis mode and paper mode only. Treat every finding as if real money depends on it.

Context: This project connects an AI agent to Robinhood's official Agentic Trading MCP (<https://agent.robinhood.com/mcp/trading>), HTTP transport, OAuth on desktop) to trade equities and options. My goal is a bounded agent: it can execute the order types I specify, and everything else is blocked by default. Guardrails must live in code, never in a prompt.

Phase 1 — Map the codebase. Walk the full file tree. Summarize the architecture, entry points, and what each module does. Explicitly identify the new dashboard files and describe what they display and what they can trigger. List every place the code can place, modify, or cancel an order.

Phase 2 — Security audit. Scan current files and full git history for committed secrets, tokens, keys, or `.env` files (`git log -p | grep -iE "token|secret|key|password"`). Confirm `.env` is gitignored. Report how OAuth tokens are stored and refreshed and flag any plaintext credential storage.

Phase 3 — Safety audit. For each item, report PRESENT / MISSING / PARTIAL with the file and line:

- Single fail-safe switch between paper and live that defaults to PAPER if the flag is missing or ambiguous
- Order verification via order history, not just an API success code
- Client-side ref-ID idempotency so retries can't double-send
- Duplicate-order dedup window
- Enforced-in-code limits: max position size, max order size, max daily loss, allowed-ticker allowlist
- Kill switch / circuit breaker on N consecutive errors or a loss threshold
- Mandatory human confirmation on all options orders

- Stale-data guard: refuse to trade on quotes older than a set threshold
- Options specifics: correct strike/expiry selection, atomic multi-leg submission (never leg-by-leg), assignment/decay awareness
- Audit logging of every decision and order

Phase 4 — Integration plan. Propose (do not execute) how to: wire the official Robinhood MCP into this project; expose my strategy logic ("trading brain") as the decision layer that calls the MCP tools; and insert a guardrail module that sits between the brain and every order call, enforcing the Phase 3 limits in code so no order reaches Robinhood without passing them. Show where confirmation prompts for options orders belong. Recommend how to incorporate skill patterns from `robinhood-for-agents` (tool/workflow structure), `open-stocks-mcp` (robust transport/error handling), and `robinhood-cli-mcp-api` (idempotency, dedup, order-evidence) — adapted, not copied, and preferring the official MCP endpoint over any private-API calls.

Phase 5 — Test plan. Lay out the sequence to validate before any live funding: read-only exploration, paper-mode strategy runs, and deliberate guardrail-breach tests that confirm each limit actually blocks an order. Do not proceed past paper mode.

Output a prioritized findings list (Critical / High / Medium), then the integration plan, then the test plan.

5. Suggested build order

1. Read-only MCP first — learn the tool surface with zero execution risk.
 2. Build strategy and guardrails against the paper-trading harness.
 3. Verify guardrails actually fire — deliberately try to breach a limit, confirm it blocks.
 4. Connect the official agentic MCP with a tiny funded amount.
 5. Only then consider scheduling / always-on automation (highest risk — last, not first).
-

6. Review checklist (reference)

Secrets and auth: no hardcoded credentials; check git history; `.env` gitignored; secure OAuth token storage and refresh.

Paper vs. live: single fail-safe switch defaulting to paper; provable simulation vs. real send via order history.

Order flow: trace one full path (signal → sizing → construction → submission → confirmation → logging); verify fills against history; ref-ID idempotency; duplicate protection.

Guardrails: max position size, max daily loss, max order size, allowed-ticker allowlist — all in code; kill switch; mandatory confirmation on options; stale-data refusal.

Options-specific: correct strike/expiry selection; atomic multi-leg submission; assignment/decay awareness.

Robustness: rate limiting and timeout handling; full audit logging; graceful handling of dropped MCP connection.